DOCUMENTAÇÃO · VISÃO GERAL

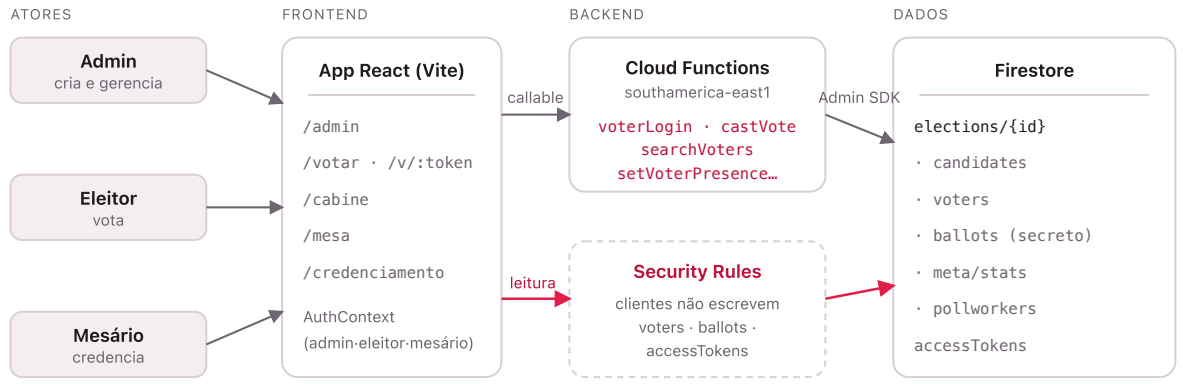
Como funciona o Suffra

Plataforma de eleições com voto secreto e credenciamento de eleitores. Duas jornadas — votar e confirmar presença — sobre a mesma base React + Firebase, com o que é sensível sempre passando por Cloud Functions.

---

01 A arquitetura em camadas

Três perfis usam o mesmo app. Escritas sensíveis (login, voto, busca, presença) vão para as Cloud Functions, que falam com o Firestore pelo Admin SDK. O cliente só toca o banco direto para *leituras* — e, quando o evento habilita, para gravar presença offline — sempre sob as Security Rules.



O gargalo é proposital. Nada sensível é escrito pelo cliente: o app pede às Cloud Functions, que validam e gravam. As Security Rules barram acesso direto a `ballots`, `accessTokens` e à escrita de `voters` — a exceção controlada é a presença offline (seção 04).

Leituras otimizadas

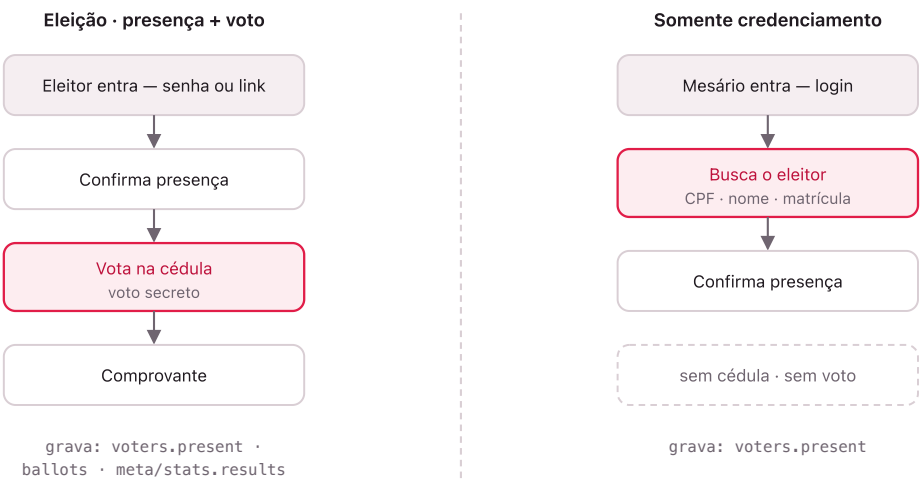
Os painéis escutam só `meta/stats` — um agregado mantido pelas functions. Nunca somam votos varrendo coleções.

Segredos nunca em claro

Senhas viram hash scrypt; links viram SHA-256. O valor legível só existe na resposta do import.

02 Dois tipos de evento

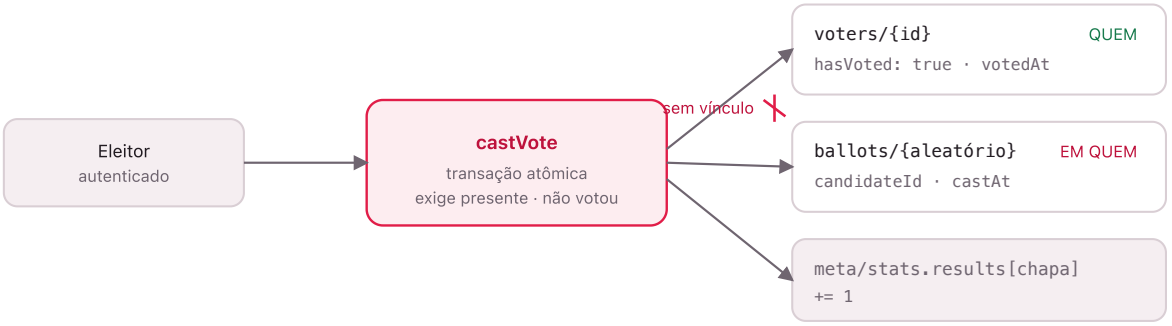
Ao criar, o admin escolhe o tipo. A diferença está no ator e no destino: na *eleição*, o próprio eleitor entra e vota; no *credenciamento*, o mesário busca e confirma — sem cédula.



A diferença é o que o credenciamento não tem. Sem chapas, sem cédula, sem `ballots` — o eleitor nem faz login. O passo que distingue cada jornada está destacado: votar de um lado, buscar do outro.

03 Como o voto fica secreto

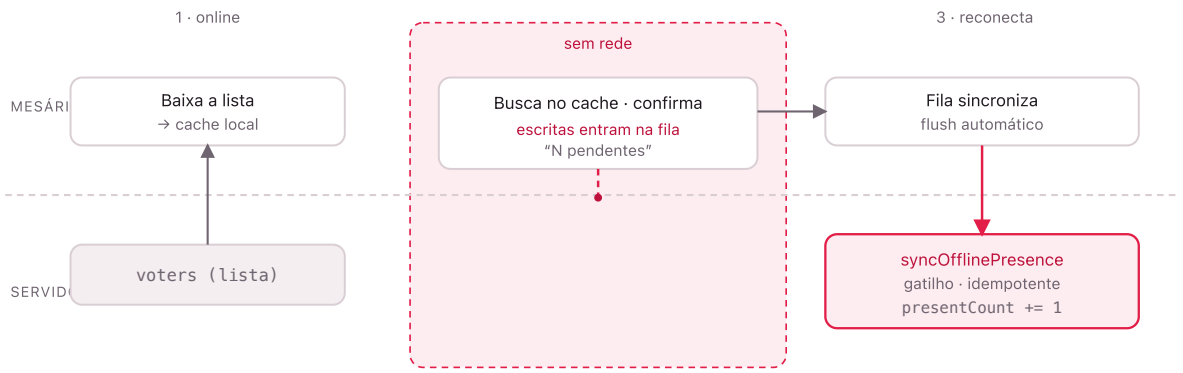
Uma única transação registra *que* a pessoa votou e, separadamente, *em quem* — sem nada ligando os dois. A garantia de 1 eleitor = 1 voto vive na transação, não numa referência entre os registros.



O elo que não existe é a segurança. A cédula tem id aleatório e guarda só o candidato e a hora; o flag `hasVoted` fica no eleitor. Não há campo que ligue uma coisa à outra — nem o admin consegue reconstruir em quem alguém votou.

04 Credenciamento com internet instável

Opcional por evento. Quando ligado, a cabine do mesário fala direto com o Firestore: a busca roda no cache e as confirmações entram numa fila que sincroniza sozinha ao reconectar. O contador é mantido por um gatilho idempotente — reenvio não conta em dobro.



A fila do SDK faz o trabalho. Enquanto cai, a confirmação fica marcada como pendente no aparelho; ao voltar, sincroniza e o gatilho `syncOfflinePresence` ajusta o `presentCount`. Como ele reage à mudança do campo `present`, reconfirmar alguém já presente resulta em delta zero — nunca conta duas vezes. Eventos sem a opção seguem 100% via Cloud Functions.

05 O modelo de dados

Tudo pendura sob cada evento. As anotações mostram quem pode ler ou escrever cada coleção.

```
admins/{uid} — só leitura do próprio doc · bootstrap por script
elections/{id} name · status(draft·open·closed) · kind(election·attendance)
· offlinePresence?
  candidates/{id} nome · número · ordem
  voters/{id} nome · cpf? · matrícula · nameLower/nameTokens · present ·
  hasVoted
    ↳ escrita só via Functions — exceto presença offline (mesário, campos
    de presença)
  ballots/{aleatório} candidateId · castAt — cliente nunca lê nem escreve ·
  voto secreto
  meta/stats totalVoters · presentCount · votedCount · results{ } —
  agregado das Functions
  pollworkers/{id} nome · login · hash da senha
accessTokens/{sha256} → eleitor — só Functions · modo link
```

Suffra — React · Vite · TypeScript · Tailwind · Firebase (Auth · Firestore · Cloud Functions · Hosting). Região das functions: `southamerica-east1`.